PROJECT SYNOPSIS

1. Final Project Title

Performance Analysis of Parallel N-Body Gravitational Simulation Using OpenMP, MPI and CUDA

2. Team Member Details

Member 1: Athul John Asish

Register No.: 240905474

Member 2: Raghav Ramprasath

Register No.: 240905544

3. Problem Statement

The N-body problem involves calculating the gravitational interaction between a large number of particles or celestial bodies. For every particle, the force exerted by every other particle must be calculated, resulting in $O(N^2)$ computational complexity. As the number of particles increases, the computation becomes increasingly time-consuming.

The project aims to implement a parallel N-body gravitational simulation using OpenMP, MPI and CUDA and analyze the performance of different parallel computing approaches against a sequential implementation.

4. Motivation for Selecting the Problem

N-body simulation is an important computational problem in astronomy and astrophysics, where simulations can involve a very large number of interacting bodies.

The direct N-body approach requires a large number of pairwise calculations, making it suitable for studying parallel computing and scalability. The force calculations for different particles can be performed independently, allowing the workload to be distributed using shared-memory CPU threads, distributed-memory processes, and GPU threads.

The motivation of this project is to investigate how different parallel computing approaches improve the performance of an $O(N^2)$ N-body simulation and to determine the strengths and limitations of each approach.

5. Objectives of the Project

1. Implement an N-body gravitational simulation where each particle interacts with other particles.
2. Develop a sequential implementation to establish a performance baseline.
3. Develop an OpenMP implementation using multiple CPU threads.
4. Develop an MPI implementation by distributing the computational workload among multiple processes.
5. Develop a CUDA implementation to perform parallel computation on an NVIDIA GPU.
6. Compare the performance of sequential, OpenMP, MPI and CUDA implementations.
7. Evaluate the effect of different numbers of particles on execution time.
8. Evaluate the effect of different numbers of threads/processes and GPU configuration on performance.
9. Calculate execution time, speedup, parallel efficiency and scalability.
10. Verify the correctness of the parallel implementations by comparing their final particle states with the sequential implementation.

6. Parallel Programming Model(s) to be Used (OpenMP/MPI/CUDA/Hybrid)

OpenMP: Will be used for shared-memory CPU parallelization. The computationally expensive force-calculation loop will be distributed among multiple CPU threads using #pragma omp parallel for. The particle position-update loop can also be parallelized because each particle can be updated independently.

MPI: Will be used for distributed-memory parallelization. The particles and force-calculation workload will be divided among multiple processes. MPI communication will be used where processes need to exchange the required particle information and combine the results.

CUDA: Will be used for GPU-based parallelization. The force calculations can be assigned to many GPU threads, allowing a large number of particle interactions to be processed concurrently on an NVIDIA GPU.

7. Proposed Methodology

Step 1 – Generate Particle Dataset

Each particle contains:

- Mass
- 3D position: x, y, z
- 3D velocity: vx, vy, vz

The primary dataset for this project will be the publicly available Star Cluster Simulations dataset, which contains the output of a real direct N-body simulation of a star cluster of 64,000 stars, generated using a King model. Each snapshot file provides the mass, 3D position (x, y, z), and 3D velocity (vx, vy, vz) of every star at a given simulation time, expressed in standard N-body units. The initial snapshot will be reformatted to match the input structure required by the sequential, OpenMP, MPI and CUDA implementations, giving the project a real-world particle distribution rather than an artificial one.

To evaluate performance and scalability across a wider range of particle counts, this real dataset will be supplemented with existing astronomical N-body datasets of varying size (e.g., the Princeton COS 126 N-Body Simulation datasets, such as planets.txt, galaxy10k.txt, galaxy20k.txt and galaxy30k.txt).

In addition, a randomly generated particle dataset (mass, position and velocity assigned programmatically) will be used during development for controlled correctness testing and debugging of the sequential and parallel implementations at small particle counts, before the real datasets are used for the main performance experiments.

Step 2 – Sequential Simulation

The sequential program will:

- Read the particle data.
- Calculate the gravitational interaction between particles.
- Update the velocity of each particle.
- Update the position of each particle.
- Repeat the process for multiple simulation iterations.
- Store the final particle state.

Step 3 – OpenMP Implementation

The computationally expensive force calculation will be parallelized using OpenMP. Different CPU threads will calculate the forces for different particles simultaneously. The position-update loop will also be parallelized.

Step 4 – MPI Implementation

The particle workload will be distributed among multiple MPI processes. Each process will perform force calculations for its assigned particles, with MPI communication used to exchange required data between processes.

Step 5 – CUDA Implementation

The N-body computation will be mapped onto GPU threads. CUDA will be used to exploit the large number of parallel processing units available on the GPU.

Step 6 – Performance Experiments

The implementations will be tested using different numbers of particles and different parallel configurations.

Example particle counts: 1,000, 5,000, 10,000, 20,000, etc.

Example CPU thread/process counts: 1, 2, 4, 8, etc.

CUDA experiments will use suitable GPU thread/block configurations based on the available hardware.

Step 7 – Performance Analysis

The execution time of all implementations will be measured.

Speedup will be calculated as:

Speedup = Sequential Execution Time / Parallel Execution Time

Parallel efficiency will be calculated as:

Efficiency = Speedup / Number of Processing Units

The results will be presented using tables and graphs.

8. Expected Outcomes

- A working N-body gravitational simulation.
- A sequential implementation serving as the performance baseline.
- OpenMP, MPI and CUDA parallel implementations.
- Reduced execution time compared with the sequential implementation for suitable workloads.
- Experimental comparison of CPU shared-memory, distributed-memory and GPU parallelization.

- Analysis of speedup, efficiency and scalability.
- Demonstration of the $O(N^2)$ computational cost of the direct N-body algorithm.
- Identification of how particle count affects computational cost.
- Identification of how increasing CPU threads/processes or GPU parallelism affects performance.
- Verification that the parallel implementations produce correct results compared with the sequential implementation.

9. Tentative Implementation Plan

11. Phase 1: Study the N-body gravitational simulation and pairwise force calculation.
12. Phase 2: Generate and prepare the particle dataset.
13. Phase 3: Implement and test the sequential N-body simulation.
14. Phase 4: Implement OpenMP parallelization.
15. Phase 5: Implement MPI parallelization.
16. Phase 6: Implement CUDA parallelization.
17. Phase 7: Test different particle counts and parallel configurations.
18. Phase 8: Measure execution time and calculate speedup and efficiency.
19. Phase 9: Compare outputs from sequential, OpenMP, MPI and CUDA implementations for correctness.
20. Phase 10: Generate graphs, analyze scalability, and compare the three parallel approaches.
21. Phase 11: Prepare the final report, documentation and presentation.